

Bulk Product Upload

How to import 30,000 (or 100,000) products from one Excel file — explained step by step, with simple code.

Stack: .NET 8 · PostgreSQL · Kafka · Angular | Audience: developers learning how it works internally

1. The problem (why the naive way fails)

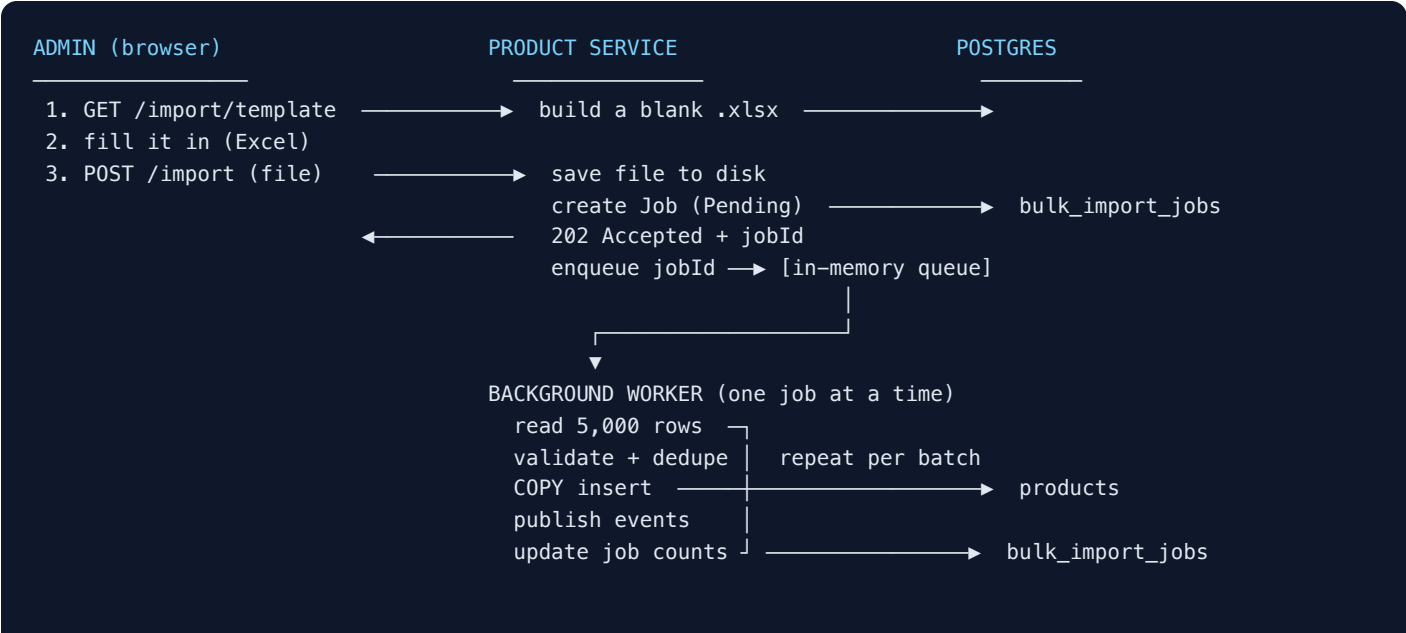
The obvious approach — "loop over the rows and save each product" — collapses at 30K rows. Three things break:

Naive approach	Why it fails at 30K–100K
Loop + <code>SaveChanges()</code> per row	30,000 separate <code>INSERT</code> round-trips to the DB. Minutes of waiting.
Add all, then one <code>SaveChanges()</code>	EF's change-tracker holds 30,000 objects in memory; one bad row rolls back <i>everything</i> .
Do it all inside the HTTP request	The browser/gateway times out long before it finishes.
Fire one Kafka event per product	30,000 awaited network calls — the Inventory service gets hammered.

So the design has to answer three questions: **(a)** how do we not block the HTTP request, **(b)** how do we insert fast, and **(c)** how do we report bad rows without failing the whole file.

The answer in one line: accept the file and return immediately with a *job id* (HTTP 202); a background worker streams the file in *batches*, inserts each batch with PostgreSQL `COPY`, and records progress so the user can poll and download an error report.

2. The big picture



- 4. GET /import/{jobId} —————> return live status (processed/ok/failed)
- 5. GET /import/{jobId}/errors —————> return .xlsx of rejected rows

3. The Excel template

The admin first downloads a template so the columns are exactly what the parser expects. One sheet named `Products` with a header row:

Name	Description	Sku	Price	Category	ImageUrl	InitialStock
Wireless Mouse	2.4GHz mouse	WM-001	19.99	Accessories	https://.../wm.jpg	150

Required: Name, Sku, Price, Category, InitialStock. **Optional:** Description, ImageUrl.

4. Step-by-step implementation

Step 1 — Upload endpoint: take the file, return a job id

The controller does almost no work. It validates the file is an `.xlsx`, hands it to the service, and returns `202 Accepted` with the job id. The heavy lifting happens later, in the background.

```
[HttpPost("/api/products/import")]
[Authorize(Roles = "Admin")]
public async Task<IActionResult> Upload(IFormFile file, CancellationToken ct)
{
    if (file is null || !file.FileName.EndsWith(".xlsx"))
        throw new ValidationException(["Please upload an .xlsx file."]);

    using var stream = file.OpenReadStream();
    var job = await _import.CreateJobAsync(stream, file.FileName, ct);

    // 202 = "I accepted it, processing happens asynchronously"
    return Accepted(new { jobId = job.JobId });
}
```

Step 2 — Create the job: save the file, write a job row, enqueue

`CreateJobAsync` persists the workbook (so the worker can re-open it after the request ends), writes a `Pending` job row to track progress, and drops the job id onto an in-memory queue.

```
public async Task<BulkImportJob> CreateJobAsync(Stream file, string name, CancellationToken ct)
{
    var job = BulkImportJob.Create(name);           // status = Pending

    await _fileStore.SaveUploadAsync(job.Id, file, ct); // save before enqueue!
    await _jobs.AddAsync(job, ct);
    await _uow.SaveChangesAsync(ct);

    await _queue.EnqueueAsync(job.Id, ct);          // wake the worker
    return job;
}
```

Order matters: save the file *before* enqueueing, so the worker can never pick up a job whose file isn't on disk yet.

Step 3 — The queue and the worker

The queue is just a `.NET Channel<Guid>` — an in-memory producer/consumer pipe. A `BackgroundService` reads job ids off it and processes them one at a time.

```
// The queue (singleton)
private readonly Channel<Guid> _ch = Channel.CreateUnbounded<Guid>();
public ValueTask EnqueueAsync(Guid id, CancellationToken ct) => _ch.Writer.WriteAsync(id, ct);
public ValueTask<Guid> DequeueAsync(CancellationToken ct) => _ch.Reader.ReadAsync(ct);

// The worker (hosted background service)
protected override async Task ExecuteAsync(CancellationToken stop)
{
    while (!stop.IsCancellationRequested)
    {
        var jobId = await _queue.DequeueAsync(stop); // blocks until a job arrives
        using var scope = _scopeFactory.CreateScope(); // fresh DbContext per job
        var service = scope.ServiceProvider.GetRequiredService<IProductImportService>();
        await service.ProcessJobAsync(jobId, stop); // do the real work
    }
}
```

Why one at a time? A 100K-row import shouldn't starve the service's request threads or exhaust the DB connection pool. Processing jobs sequentially keeps the service responsive.

Step 4 — Process the job in batches (the core loop)

This is the heart of it. We **stream** the rows (never load all 30K into memory), gather them into batches of 5,000, and process each batch independently. After every batch we update the job counts so polling shows live progress.

```
public async Task ProcessJobAsync(Guid jobId, CancellationToken ct)
{
    var job = await _jobs.GetByIdAsync(jobId, ct);
    job.Start(); // status = Processing
    await _uow.SaveChangesAsync(ct);

    var errors = new List<ImportError>();
    var seenSkus = new HashSet<string>(); // detect duplicates within the file

    using var file = await _fileStore.OpenUploadAsync(jobId, ct);

    foreach (var batch in Chunk(_parser.Parse(file), size: 5000))
    {
        var (inserted, batchErrors) = await ProcessBatch(batch, seenSkus, ct);
        errors.AddRange(batchErrors);

        job.RecordBatch(inserted, batchErrors.Count); // update counters
        await _uow.SaveChangesAsync(ct); // → poller sees progress
    }

    if (errors.Count > 0)
        await _fileStore.SaveErrorReportAsync(jobId, _parser.BuildErrorReport(errors), ct);

    job.Complete(hasErrors: errors.Count > 0);
    await _uow.SaveChangesAsync(ct);
}
```

What "streaming + chunking" buys you: memory stays flat. Whether the file has 3K or 300K rows, we only ever hold ~5,000 parsed rows at once.

Step 5 — Parse the Excel rows (lazily)

We use ClosedXML (a free, MIT-licensed library) to read the workbook. `Parse` returns an `IEnumerable` with `yield return`, so rows are produced one at a time as the loop asks for them — not all up front.

```
public IEnumerable<ProductRow> Parse(Stream workbook)
{
    using var wb = new XLWorkbook(workbook);
    var sheet = wb.Worksheet("Products");

    foreach (var row in sheet.RowsUsed().Skip(1)) // skip the header
    {
        yield return new ProductRow
        {
            RowNumber = row.RowNumber(),
            Name       = row.Cell(1).GetString(),
            Sku        = row.Cell(3).GetString(),
            Price      = row.Cell(4).GetValues<decimal>(),
            Category   = row.Cell(5).GetString(),
            Stock      = row.Cell(7).GetValues<int>(),
            // ...
        };
    }
}
```

Step 6 — Validate & remove duplicates (per batch)

For each batch we (1) validate every row, (2) drop rows whose SKU repeats earlier in the file, and (3) drop rows whose SKU already exists in the catalog. Every dropped row is recorded with its row number and reason — the batch is never aborted.

```
private async Task<(int inserted, List<ImportError>)> ProcessBatch(
    IReadOnlyList<ProductRow> rows, HashSet<string> seenSkus, CancellationToken ct)
{
    var errors = new List<ImportError>();
    var candidates = new List<ProductRow>();

    foreach (var row in rows)
    {
        if (!_validator.Validate(row).IsValid) // bad data
            { errors.Add(new(row.RowNumber, row.Sku, "Invalid data")); continue; }

        var sku = row.Sku.ToUpperInvariant();
        if (!seenSkus.Add(sku)) // duplicate in the file
            { errors.Add(new(row.RowNumber, row.Sku, "Duplicate in file")); continue; }

        candidates.Add(row);
    }

    // One query: which of these SKUs already exist in the DB?
    var existing = await _inserter.GetExistingSkusAsync(candidates.Select(c => c.Sku), ct);

    var products = new List<Product>();
    foreach (var row in candidates)
    {
        if (existing.Contains(row.Sku.ToUpperInvariant()))
            errors.Add(new(row.RowNumber, row.Sku, "SKU already exists"));
        else
            products.Add(Product.Create(row.Name, row.Sku, row.Price, /*...*/));
    }

    await _inserter.BulkInsertAsync(products, ct); // Step 7
    await _events.PublishCreatedBatchAsync(products, ct); // Step 8
    return (products.Count, errors);
}
```

Step 7 — The fast insert: PostgreSQL COPY

This is the speed secret. Instead of thousands of `INSERT` statements, PostgreSQL's `COPY` command streams rows straight into the table in a single binary operation. Npgsql exposes it via `BeginBinaryImport`.

```
public async Task BulkInsertAsync(IReadOnlyCollection<Product> products, CancellationToken ct)
{
    var conn = (NpgsqlConnection)_db.Database.GetDbConnection();
    await conn.OpenAsync(ct);

    await using var writer = await conn.BeginBinaryImportAsync(
        "COPY products (id, name, sku, price, category, stock, created_at) " +
        "FROM STDIN (FORMAT BINARY)", ct);

    foreach (var p in products)
    {
        await writer.StartRowAsync(ct);
        await writer.WriteAsync(p.Id,           NpgsqlDbType.Uuid, ct);
        await writer.WriteAsync(p.Name,         NpgsqlDbType.Text, ct);
        await writer.WriteAsync(p.Sku,          NpgsqlDbType.Text, ct);
        await writer.WriteAsync(p.Price,        NpgsqlDbType.Numeric, ct);
        await writer.WriteAsync(p.Category,     NpgsqlDbType.Text, ct);
        await writer.WriteAsync(p.Stock,        NpgsqlDbType.Integer, ct);
        await writer.WriteAsync(p.CreatedAt,    NpgsqlDbType.TimestampTz, ct);
    }

    await writer.CompleteAsync(ct); // nothing is written until this line
}
```

Method	~Time for 100K rows	Notes
EF loop + SaveChanges each	several minutes	100K round-trips
EF AddRange + one SaveChanges	tens of seconds	heavy memory
PostgreSQL COPY	a few seconds	binary stream, minimal memory

The trade-off: COPY bypasses EF and aborts the whole batch if any row violates the unique SKU index. That's exactly why Step 6 pre-removes duplicates — by the time we COPY, every row is known-good.

Step 8 — Tell the rest of the system (batched events)

Other services (Inventory) need to know about the new products. Instead of `await`-ing one Kafka send per product, we fire them all non-blocking and `Flush` once per batch.

```
public Task PublishCreatedBatchAsync(IEnumerable<Product> products, CancellationToken ct)
{
    foreach (var p in products)
        _producer.Produce("product-created", ToMessage(p)); // non-blocking enqueue

    _producer.Flush(ct); // wait once for the whole batch to be delivered
    return Task.CompletedTask;
}
```

Step 9 — Poll status & download the error report

While the worker runs, the UI polls the status endpoint every ~1.5s. When it finishes, if any rows were rejected, the admin downloads an Excel report listing each bad row and why.

```
[HttpGet("/api/products/import/{jobId}")]
public async Task<IActionResult> Status(Guid jobId)
    => Ok(await _import.GetJobAsync(jobId));
    // → { status: "Processing", processed: 15000, succeeded: 14990, failed: 10 }

[HttpGet("/api/products/import/{jobId}/errors")]
public async Task<IActionResult> Errors(Guid jobId)
    => File(await _import.GetErrorReportAsync(jobId), XlsxType, "errors.xlsx");
```

On the Angular side, a simple timer polls until the status is terminal:

```
timer(0, 1500).pipe(
    switchMap(() => this.api.get(`/api/products/import/${jobId}`)),
    takeWhile(job => ![ 'Completed', 'CompletedWithErrors', 'Failed' ].includes(job.status), true),
).subscribe(job => this.job.set(job)); // updates the progress bar live
```

5. Why each decision was made

Decision	Reason
Return 202 + job id (async)	30K rows can't finish inside one HTTP request without timing out.
Background worker + queue	Frees the web request instantly; processes large files reliably off the request thread.
Stream + 5K batches	Constant memory regardless of file size; progress visible after each batch.
PostgreSQL COPY	Seconds instead of minutes; the only insert method that truly scales to 100K.
Skip & report duplicates/invalids	One bad row out of 30K should not fail the other 29,999.
Batched Kafka events	Avoids 30K blocking network calls to downstream services.

6. Gotchas worth remembering

- **Stable sort for paging:** when you later list these products with pagination, always order by a unique tiebreaker (e.g. `ORDER BY name, id`) or pages can duplicate/skip rows.
- **Imported products are drafts:** they start unpublished, so they don't appear in the storefront until an admin publishes them. This is intentional (review before going live).
- **COPY is all-or-nothing per batch:** always pre-clean the batch (dedupe + validate) before COPY.
- **In-memory queue is not durable:** if the service restarts, a queued-but-not-started job stays `Pending`. For full durability you'd re-enqueue `Pending` jobs on startup, or use a persistent queue.
- **Insert + event aren't one transaction:** a crash between COPY and publish could leave downstream services missing those SKUs. The consumer is idempotent, so re-publishing is the recovery path (a transactional outbox would make it bullet-proof).

CommerceSphere — Bulk Product Upload implementation guide. The code shown is simplified for clarity; the production code adds correlation ids, structured logging, cancellation handling, and full validation rules.